Technical Report: Event Prediction for Traffic Simulation

March 28, 2026

1 Purpose and Scope

This report documents a general framework for event prediction in simulated dynamic systems, with the current traffic intersection environment serving as the first concrete application space.

The report is designed to stay useful as the repository grows. In particular, it is intended to support:

- additional predictive model families
- additional simulation environments beyond traffic
- broader evaluation protocols
- eventual migration toward richer event-graph or EventFlow-like representations

The current system studies timestamped event prediction in a discrete-event traffic simulator. At a high level, each model observes a history of past events and current system state, then predicts:

- what event happens next
- when it happens

The repository currently supports both one-step evaluation and short-horizon multi-event rollout evaluation.

2 Problem Setting

2.1 Event Prediction as a Marked Temporal Process

We model the observed system as an ordered sequence of timestamped events

$$\mathcal{E} = \{(t_1, m_1), (t_2, m_2), \ldots, (t_n, m_n)\},$$

where:

- $t_i \in \mathbb{R}_{\geq 0}$ is the event time
- $m_i \in \mathcal{M}$ is the event mark or label

The mark set $\mathcal{M}$ depends on the application. In the current traffic setting, marks encode both event family and lane or phase subtype, such as:

- `phase_change:NS_GREEN`
- `vehicle_arrival:north`
- `vehicle_departure:west`

The prediction task is to model the conditional distribution

$$p(t_{i+1}, m_{i+1} \mid \mathcal{H}_i),$$

where $\mathcal{H}_i$ is the history up to and including event i. In practice, $\mathcal{H}_i$ may include:

- recent event labels
- recent inter-event times
- explicit state features derived from the environment

2.2 One-Step and Rollout Forecasting

Two related tasks are evaluated:

1. One-step forecasting. Predict the next event label and time from the true current history.
2. Multi-step rollout forecasting. Predict several future events autoregressively by feeding the model's own predictions back into its state.

These tasks are related but not identical. Strong one-step models may still drift under rollout because timing and label errors compound.

3 Application Space

3.1 General Application Template

The broader application class is event prediction in structured dynamical systems with:

- asynchronous event arrivals
- latent or explicit state
- constraints or control logic
- state transitions caused by both exogenous events and system rules

Examples beyond traffic could include:

- manufacturing systems
- computer networks
- healthcare workflows
- order fulfillment systems
- autonomous system coordination

This makes the current repository a prototype for event forecasting in controlled systems, not only for transportation.

3.2 Current Application: Signalized Traffic Intersection

The current environment is a four-way signalized intersection with:

- four inbound approaches: `north`, `south`, `east`, `west`
- a fixed-time signal controller
- stochastic arrivals
- queueing and service dynamics

The simulator is intentionally simple enough to make model behavior interpretable, while still coupling:

- control state
- demand state
- queue evolution
- event timing

That combination is useful because it creates a nontrivial forecasting problem without hiding the ground-truth mechanism.

4 Current Traffic Simulation Design

4.1 Entities and State

The current simulation can be understood as a small stateful system with the following entities:

- Intersection controller
  - current phase
  - phase index
  - phase timing
- Lane objects
  - queue length
  - total arrivals
  - total departures
  - cumulative wait
  - max queue

Operationally, the controller and lane queues are the main evolving state variables.

4.2 Event Types

The simulator currently emits three event families:

- `phase_change`
- `vehicle_arrival`
- `vehicle_departure`

These event types are sufficient to reconstruct:

- causal order
- signal state
- queue mutations
- service chaining behavior

4.3 Control Logic

The signal plan is fixed-time, cycling through:

- `NS_GREEN`
- `ALL_RED`
- `EW_GREEN`
- `ALL_RED`

When a green phase becomes active, compatible queued lanes may begin service. Departures are generated under a fixed service headway as long as:

- the lane is still green
- the queue remains nonempty

4.4 Arrival and Service Assumptions

Arrivals are generated as stochastic interarrival times using exponential sampling per lane. Departures are triggered by queue presence and a fixed saturation headway. In that sense, the simulator combines:

- exogenous stochastic arrivals
- deterministic control scheduling
- state-dependent service events

4.5 Why This Environment Matters

This design is a good first application because it has:

- explicit state
- interpretable rules
- mixed stochastic and deterministic structure
- meaningful long-horizon error accumulation

It is also a natural stepping stone toward a more explicit event/state graph representation.

5 Model Families

This section is written to remain extensible as new models are added. The current repository includes both simple baselines and learned temporal models.

5.1 Mechanistic and Empirical Baselines

5.1.1 Global Rate Baseline

Intuition: This model predicts events based on global recurrence statistics. It assumes that each event type tends to recur at a characteristic average gap, with limited dependence on current context.

Mathematical sketch: For each label m, estimate an average recurrence gap $\bar{\Delta}_m$. At prediction time, choose the label with the smallest residual expected time:

$$\hat{t}_m = t + \max(\epsilon, \bar{\Delta}_m - (t - t_m^{\text{last}})).$$

This is simple and cheap, but it ignores most system structure.

5.1.2 Transition Baseline

Intuition: This model predicts the next event from the current event using empirical transition counts and average transition delays.

Mathematical sketch: Estimate

$$\hat{m}_{i+1} = \arg\max_m \ \hat{p}(m \mid m_i),$$

and then use an empirical mean delay for the selected transition:

$$\hat{t}_{i+1} = t_i + \mathbb{E}[\Delta t \mid m_i, m_{i+1} = \hat{m}_{i+1}].$$

This is stronger than a pure global-rate model, but still only weakly state-aware.

5.1.3 Mechanistic Baseline

Intuition: This model uses explicit knowledge of queue state, phase timing, and service logic. It predicts the earliest plausible next event from controller timing, estimated arrivals, and queued departures.

Mathematical sketch: Construct candidate event times for:

- next signal phase transition
- next lane arrival for each lane
- next scheduled departure for each active queued lane

Then predict:

$$(\hat{t}, \hat{m}) = \arg \min_{(t,m)\in\mathcal{C}} t.$$

This is highly interpretable and often strong in structured systems.

5.2 Learned Neural Temporal Models

5.2.1 GRU-Based Neural TPP

Intuition: This model encodes recent event history with a recurrent network, then combines that representation with current environment state features to predict both next-event type and next-event delay.

Mathematical sketch: Given embedded event labels and inter-event times,

$$h_i = \mathrm{GRU}(h_{i-1}, x_i),$$

where x_i contains event-label and timing features. Then predict:

$$\hat{p}(m_{i+1} \mid h_i, s_i), \qquad \hat{\Delta t}_{i+1} = f(h_i, s_i),$$

where s_i is the explicit system-state feature vector.

5.2.2 Continuous-Time LSTM-Style TPP

Intuition: This model is similar in spirit to the GRU predictor, but it explicitly decays hidden state as a function of elapsed time. That makes it more natural for irregular event timing.

Mathematical sketch: Let the hidden state decay between events:

$$h_i^- = g(\Delta t_i) \odot h_{i-1},$$

then update with a recurrent transition and predict the next event and delay from the resulting state.

This gives the model a continuous-time inductive bias without requiring a full intensity-based point process formulation.

5.2.3 Transformer-Style Temporal Point Process

Intuition: This model uses attention over recent event history instead of relying only on recurrence. It can use broader context when the next event depends on patterns spanning multiple earlier events.

Mathematical sketch: Encode a sequence of event embeddings and timing features,

$$z_{1:i} = \mathrm{Transformer}(x_{1:i}),$$

then use a prediction token or pooled representation to predict:

$$\hat{p}(m_{i+1} \mid z_{1:i}, s_i), \qquad \hat{\Delta t}_{i+1} = f(z_{1:i}, s_i).$$

The current implementation also includes rollout-aware scheduled-sampling style training to reduce autoregressive drift.

6 Mathematical View of the Learning Objective

The learned models in the current repository do not yet optimize a full event-sequence likelihood. Instead, they solve a supervised next-event prediction problem with two heads:

- classification head for event type
- regression head for next-event delay

The objective can be written generically as

$$\mathcal{L} = \mathcal{L}_{\text{type}} + \lambda \mathcal{L}_{\text{time}},$$

where:

- $\mathcal{L}_{\text{type}}$ is cross-entropy over event labels
- $\mathcal{L}_{\text{time}}$ is a robust regression loss on log-transformed delay
- λ controls the tradeoff between label accuracy and timing accuracy

For the rollout-aware transformer variant, an auxiliary loss is added on a later predicted step after partially rolling the model forward on its own predictions. Conceptually:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{1-step}} + \alpha \mathcal{L}_{\text{rollout}},$$

where α is a smaller weight.

7 State Representation

The current feature representation provided to the learned models includes:

- per-lane queue features
- phase indicator features
- normalized elapsed phase time
- departure-due features for each lane
- recent event labels
- recent inter-event times

This is important because the repository is not solving an event-only language-model problem. It is solving event prediction in a controlled stateful system.

8 Experimental Design

8.1 Data Generation

Training and evaluation runs are generated from the traffic simulator with different seeds. Each run produces:

- an event trace
- aggregate summary statistics

The current benchmark uses the generated traces directly as supervised learning data.

8.2 Metrics

The current evaluation focuses on event-centric forecasting metrics:

- one-step type accuracy
- one-step family accuracy
- one-step time MAE
- one-step time RMSE
- one-step within-2-seconds accuracy
- rollout type accuracy
- rollout family accuracy
- rollout time MAE

Type accuracy is exact label matching. Family accuracy evaluates whether the model got the event family right even if the lane or phase subtype was wrong.

8.3 Dashboarding

The repository also includes a browser dashboard for comparing:

- aggregate metric tables
- short-horizon and rollout performance
- time error over prediction steps
- prediction versus actual next-event timing

This is useful because event forecasting quality is multidimensional: a model may be strong on timing but weaker on exact event identity, or vice versa.

9 Current Results

The latest stored benchmark in `analysis/model_comparison.json` reports the following one-step metrics:

Model	Type Acc	Family Acc	Time MAE	Time RMSE	Within 2s
`global_rate`	0.1040	0.4301	0.6258	0.9220	0.9758
`transition`	0.3492	0.5947	0.4564	0.6220	0.9926
`mechanistic`	0.4967	0.5268	0.5899	1.0050	0.9524
`neural_tpp`	0.4953	0.7145	0.3954	0.5707	0.9933
`continuous_tpp`	0.4911	0.7016	0.3950	0.5561	0.9940
`transformer_tpp`	0.4669	0.6641	0.4013	0.5885	0.9923

The corresponding 10-event rollout metrics are:

Model	Rollout-10 Type Acc	Rollout-10 Family Acc	Rollout-10 Time MAE
`global_rate`	0.1121	0.4706	1.7948
`transition`	0.1681	0.4922	1.0144
`mechanistic`	0.2390	0.4479	3.9851
`neural_tpp`	0.2511	0.5284	1.0401
`continuous_tpp`	0.2447	0.5195	0.9603
`transformer_tpp`	0.2429	0.5312	1.0938

10 Interpretation of Results

Several patterns are already visible:

10.1 Learned models improve event-family discrimination

The learned temporal models outperform the simple empirical baselines on family accuracy, especially at one step. This suggests that recent history plus explicit system state is informative in a way that simple transition tables do not capture fully.

10.2 Continuous-time inductive bias helps timing

The continuous-time LSTM-style model is especially strong on timing error. That is consistent with the intuition that irregular event spacing benefits from explicit time-aware state evolution.

10.3 Exact type prediction remains competitive for mechanistic models

The mechanistic model remains strong on exact event type accuracy because it encodes the simulator's structure directly. However, its rollout timing error grows sharply, which suggests that approximate arrival modeling and rigid event-choice logic accumulate error over longer horizons.

10.4 Rollout remains much harder than one-step prediction

All models degrade under 10-event rollout. This is expected because the evaluation becomes autoregressive, so label and timing errors feed back into future predictions.

11 Limitations

The current system has several important limitations:

- it uses one simulation environment only
- the simulator is intentionally simple
- the learned models are supervised next-event predictors rather than full likelihood-based temporal point processes
- rollout evaluation is still relatively short horizon
- no explicit uncertainty calibration is reported yet

These limitations are acceptable for a first benchmark, but they matter when interpreting performance differences.

12 Extension Paths

This report structure is intended to expand naturally.

12.1 New Models

Future model sections can be added for:

- Neural Hawkes models
- Transformer Hawkes variants
- flow-based temporal point processes
- diffusion or flow-matching trajectory generators
- state-space sequence models

12.2 New Application Spaces

The application-space section can be extended to additional environments, for example:

- logistics
- production systems
- scheduling systems
- networked cyber-physical systems

Each application can reuse the same high-level template:

- entities and state
- event types
- control or physics logic
- prediction targets
- evaluation metrics

12.3 Richer Evaluation

The evaluation section can later include:

- calibration metrics
- probabilistic scoring rules
- long-horizon trajectory quality
- intervention-aware evaluation
- robustness under distribution shift

13 Conclusion

The current repository establishes a solid first benchmark for event prediction in a structured simulated system. The traffic-intersection environment provides a controlled but nontrivial testbed where:

- mechanistic reasoning matters
- timing matters
- autoregressive rollout matters

The present results show that learned temporal models already improve over simple empirical baselines, while mechanistic models remain valuable reference points. The framework is therefore well positioned to support broader work on:

- stronger temporal point process models
- richer simulation environments
- eventual EventFlow-like event/state abstractions

14 Suggested Future Report Sections

To keep this document extensible, future versions may add:

- reproducibility and configuration
- implementation details and software architecture
- ablation studies
- failure cases and qualitative examples
- cross-application comparisons
- uncertainty and calibration analysis